

OpenCV= OpenCV (Open Computer Vision), bilgisayarlara görüntüleri analiz etmeyi ve anlamlandırmayı öğreten güçlü bir kütüphanedir. Görüntü işleme, bilgisayarlı görü ve yapay zekâ uygulamalarının temel taşıdır. İnsan gözü ve beyni gibi, bilgisayar da sensörlerden veya dosyalardan aldığı veriyi işler, pikselleri analiz eder ve sonuç çıkarır. OpenCV hem Python hem de C++ tarafında hızlı çalışabilen esnek bir kütüphanedir.

Görüntü Okuma Kaydetme Gösterme

OpenCV ile görüntü işlemenin ilk adımı görüntüyü kod ortamına aktarmaktır. Görüntü rOpenCV tarafından NumPy matrisleri olarak işlenir her piksel sayısal değerlerle temsil edilir.

Okuma: Görüntü bir matris hâline gelir ve üzerinde matematiksel veya mantıksal işlemler yapılabilir.

Kaydetme: İşlenmiş görüntüler diske yazılır; bu adım genellikle veri üretimi veya sonuç saklama için kritik.

Gösterme: Görüntüyü ekranda görüntülemek mümkündür, ancak sunucu veya terminal ortamlarında her zaman güvenilir olmayabilir.

Renk Uzayları – BGR, RGB, GRAY, HSV

OpenCV de renk paletleri görüntü işleme mantığının temel taşlarından biridir

BGR: OpenCV nin varsayılan formatı, çoğu Python kütüphanesinin RGB formatıyla ters sıradadır.

RGB: Görselleştirme veya başka kütüphanelerle uyum için gereklidir.

Gri Tonlama (GRAY): Kenar tespiti, eşikleme ve histogram işlemleri gri tonlamada daha verimli çalışır.

HSV (Hue, Saturation, Value): İnsan gözüne daha yakın renk algısı sağlar. Renk temelli nesne tespiti için idealdir. HSV de renk, doygunluk ve parlaklık ayrı ayrı yönetilebilir.

Görüntüyü Yeniden Boyutlandırma (Resize)

Gerçek zamanlı uygulamalarda yüksek çözünürlüklü görüntüler işlemciyi yorar.

Bu nedenle görüntüler küçültülür veya gerektiğinde büyütülür.

İşlem sırasında interpolasyon teknikleri görüntü kalitesini etkiler: küçültmede farklı, büyütmede farklı yöntemler daha iyi sonuç verir. Bu adım özellikle gerçek zamanlı sistemlerde performans optimizasyonu için önemlidir.

Parlaklık, Kontrast ve Maskeleme

OpenCV görüntüleri sayısal matrisler olarak tuttuğu için maske ve parlaklık ayarlamaları sonraki işlemlerin doğruluğu için kritik öneme sahiptir.

Parlaklık artırma: Piksel değerlerinin genelini yükseltir görüntüyü daha aydınlık hâle getirir.

Kontrast artırma: Piksel değerleri arasındaki farkı büyütür detayların daha belirgin olmasını sağlar.

Maskeleme: Görüntünün yalnızca belirli bölgeleri işlenir. Örneğin belirli bir renk aralığı veya bölge üzerinde analiz yapılabilir.

Gürültü Azaltma ve Filtreleme

Gerçek görüntüler genellikle temiz değildir gürültü azaltma kenar algılama ve nesne tespiti gibi adımların doğruluğunu ciddi biçimde artırır.

Gaussian blur: Yumuşak bulanıklık sağlar kenarları çok bozmaz.

Median blur: Noktasal gürültülerini temizlemede etkilidir.

Ortalama filtre (blur): Basit hızlı ve temel gürültü temizleme sağlar.

Histogram Analizi ve Görüntü Kalitesini Arttırma

Bir görüntüdeki piksel değerlerinin dağılımına histogram denir. Histogram işlemleri görüntünün genel aydınlık ve kontrast dengesini iyileştirerek sonraki algoritmaların sağlıklı çalışmasını sağlar.

Histogram eşitleme: Tüm parlaklık değerlerini mümkün olduğunca eşit dağıtarak kontrastı artırır.

CLAHE (Sınırlı Adaptif Histogram Eşitleme): Görüntüyü küçük parçalara bölüp bölgesel eşitleme yapar. Bu yöntem aşırı parlaklık veya patlamayı önler karanlık bölgeleri açar.

Kenar Algılama

Kenar algılama nesne sınırlarını tespit etmek için temel adımdır.

Gürültü atlatıldıktan sonra uygulanır. Algoritmalar görüntüdeki parlaklık veya renk değişimlerini tespit eder çıkan kenarlar kontur bulma ve şekil analizi gibi ileri adımlar için veri sağlar.

Eşikleme (Thresholding)

Görüntüyü siyah-beyaz hâline getirerek nesneleri arka plandan ayırmak için kullanılır.Eşikleme kontur bulma ve morfolojik işlemler için ön işleme adıımıdır.

Sabit eşik: Tüm görüntüye aynı eşik uygulanır.

Adaptif eşik: Görüntünün farklı bölgelerine farklı eşikler uygulanır özellikle düzensiz ışııkta kullanışlıdır.

Morfolojik İşlemler

Morfolojik işlemler görüntü deki beyaz bölgeleri büyütme veya küçültme mantığıyla çalışır bu işlemler, özellikle maskeli veya eşiklenmiş görüntülerde nesne sınırlarını netleştirmek için kullanılır

Erode: Beyaz bölgeleri daraltır.

Dilate: Beyaz bölgeleri genişletir.

Opening (Açma): Küçük gürültüleri temizler.

Closing (Kapama): Küçük boşlukları doldurur.

Kontur Bulma ve Şekil Analizi

Konturlar nesnelerin dış sınırlarını temsil eder.Kontur bulma eşiklenmiş veya morfolojik işlenmiş görüntülerde gerçekleştirilir. Her kontur için alan çevre minimum dikdörtgen şekil sınıflandırma gibi analizler yapılabilir.Kare sayma plaka okuma ve diğer nesne tanıma işlemleri burada başlar.

Geometrik Dönüşümler

Döndürme, taşıma, ölçekleme gibi temel dönüşümler.
Perspektif dönüşümler, eğik çekilmiş belgeleri düzeltmek için kritik öneme sahiptir.
Bu dönüşümler, nesne pozisyonlarını ve görüntü oranlarını koruyarak veri hazırlamada kullanılır.

Özellik Çıkarımı (ORB, SIFT, SURF, AKAZE)

Görüntüdeki karakteristik noktaları tespit eder kullanım alanları = Görüntü eşleştirme benzer sahne tespiti, nesne tanıma ve takip algoritmaları. ORB hızlı ve ücretsiz bir çözüm olarak yaygın tercih edilir.

Template Matching

Küçük bir görüntüyü büyük bir görüntüde arama yöntemidir. Şablon eşleştirme, endüstriyel kalite kontrol ve GUI otomasyonunda kullanılır.

Video İşleme ve Gerçek Zamanlı Uygulamalar

Kameradan veya videodan kareler alınır kareler döngüde işlenir, görüntü analizi yapılır gerçek zamanlı sistemlerde FPS ve çözünürlük optimizasyonu önemlidir VideoWriter ile işlenmiş kareler tekrar video hâline getirilebilir.

Çizim ve Anotasyon

Görüntü üzerine çizgi daire dikdörtgen veya yazı eklenebilir nesne tespit sonuçlarını görselleştirmek için kritik bir adımdır.

Nesne Tespiti

Klasik yöntemler: Haar cascades (özellikle yüz tespiti)
Derin öğrenme yöntemleri = YOLO, SSD, Faster R-CNN
Derin öğrenme modelleri, hem nesneyi bulur hem sınıflandırmasını yapar.

Nesne Takibi

Nesneyi ilk karede tespit ettikten sonra, sonraki karelerde konumunu tahmin etme işidir. Özellik çıkarımı veya şablon eşleme mantığıyla çalışır.

Popüler algoritmalar =KCF, CSRT, MOSSE, MIL, TLD

Sonuç olarak OpenCV görüntü işleme ve bilgisayarlı görü alanında temel ve ileri seviyede güçlü bir araçtır. Basit filtrelerden derin öğrenmeli nesne tespitine kadar geniş bir kullanım yelpazesi vardır. Bu yazılım aracı gelişmiş işlevsel kullanışlı bir kullanım aracıdır.

Görüntü Okuma: `img = cv2.imread("resim.jpg")` # Görüntüyü dosyadan okur

Görüntü Kaydetme: `cv2.imwrite("output.png", img)` # Görüntüyü dosyaya kaydeder

Görüntü Gösterme: `cv2.imshow("Pencere", img)` # Görüntüyü ekranda gösterir
`cv2.waitKey(0)` # Pencereyi açık tutar
`cv2.destroyAllWindows()` # Açık pencereleri kapatır

Renk Uzaı Dönüşümleri: `gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)` # Gri tonlamaya çevirir
`rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` # BGR'den RGB'ye çevirir
`hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)` # HSV renk uzayına çevirir

Görüntü Boyutlandırma: `resized = cv2.resize(img, (640, 480), interpolation=cv2.INTER_AREA)` # Görüntüyü yeniden boyutlandırır

Parlaklık ve Kontrast: `brighter = cv2.add(img, 50)` # Parlaklık artırır
`contrast = cv2.addWeighted(img, 1.5, img, 0, -20)` # Kontrast artırır

Maskeleme: `masked = cv2.bitwise_and(img, img, mask=None)` # Maskeye uyan bölgeyi alır

Gürültü Azaltma / Filtreler: `blur = cv2.GaussianBlur(img, (5,5), 0)` # Gaussian bulanıklık
`median = cv2.medianBlur(img, 5)` # Median bulanıklık
`average = cv2.blur(img, (5,5))` # Ortalama bulanıklık

Histogram Eşitleme: `eq = cv2.equalizeHist(gray)` # Histogram eşitleme
`clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))`
`c1 = clahe.apply(gray)` # CLAHE ile adaptif eşitleme

Kenar Algılama (Canny): `edges = cv2.Canny(gray, 50, 150)` # Kenar tespiti

Eşikleme (Thresholding): `_, th = cv2.threshold(gray, 120, 255, cv2.THRESH_BINARY)` # Sabit eşik
`th2 = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2)` # Adaptif eşik

Morfolojik İşlemler: `kernel = np.ones((5,5), np.uint8)`
`eroded = cv2.erode(th, kernel, iterations=1)` # Erode
`dilated = cv2.dilate(th, kernel, iterations=1)` # Dilate
`opened = cv2.morphologyEx(th, cv2.MORPH_OPEN, kernel)` # Open
`closed = cv2.morphologyEx(th, cv2.MORPH_CLOSE, kernel)` # Close

Kontur Bulma: `contours, _ = cv2.findContours(th, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)` # Konturlar

Geometrik Dönüşümler: `rows, cols = img.shape[:2]`
`M = cv2.getRotationMatrix2D((cols/2, rows/2), 45, 1)`
`rotated = cv2.warpAffine(img, M, (cols, rows))` # Döndürme
`dst = cv2.warpPerspective(img, np.eye(3), (cols, rows))` # Perspektif düzeltme

Özellik Çıkarımı (ORB): `orb = cv2.ORB_create()` # ORB nesnesi oluşturur
`kp = orb.detect(img, None)` # Anahtar noktaları bulur
`kp, des = orb.compute(img, kp)` # Anahtar noktaları ve descriptor çıkarır

Template Matching: `result = cv2.matchTemplate(img, img, cv2.TM_CCOEFF_NORMED)` # Şablon eşleştirme

Video İşleme: `cap = cv2.VideoCapture(0)` # Kamerayı aç
`ret, frame = cap.read()` # Kare oku
`cap.release()` # Kamerayı kapat

Çizim ve Anotasyon: `cv2.line(img, (0,0), (100,100), (255,0,0), 2)` # Çizgi
`cv2.circle(img, (100,100), 50, (0,255,0), 3)` # Daire
`cv2.rectangle(img, (50,50), (150,150), (0,0,255), 2)` # Dikdörtgen
`cv2.putText(img, "Merhaba", (50,50), cv2.FONT_HERSHEY_SIMPLEX, 1, (255,255,255), 2)`
Yazı

Nesne Tespiti (Haar Cascades): `face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')` # Yüz tespiti için sınıflandırıcı
`faces = face_cascade.detectMultiScale(gray, 1.3, 5)` # Yüzleri bul

Nesne Takibi (Tracker): `tracker = cv2.TrackerCSRT_create()` # CSRT tracker oluştur
`ok = tracker.init(frame, (50,50,100,100))` # Takibi başlat
`ok, bbox = tracker.update(frame)` # Takip güncelleme